

Pion qTMD and PDF Measurement

Comprehensive Physics Note

Pyquda_Measurement Collaboration

May 5, 2026

Contents

1	Physics Overview	2
1.1	Transverse Momentum Dependent Distributions	2
1.2	Equal-Time Correlators on the Lattice	2
2	Quasi-PDF and Quasi-TMD: LaMET Framework	3
3	Two-Point Function	3
3.1	Physical Definition	3
3.2	Source Gamma Convention	4
3.3	Einsum Contraction Pattern	4
3.4	Boosted Smearing	4
4	Three-Point qTMD Function (Pre-Sequential-Trick)	5
4.1	Physical Correlator	5
4.2	Why the Sequential Trick is Needed	5
5	Sequential Source	5
5.1	Meson Sequential Source Construction	5
5.2	Sink-Summed Three-Point Contraction	6
6	CG qTMD Operator	6
6.1	Coordinate-Gauge Displacement	6
6.2	Transverse Direction Scan	6
6.3	Wilson Line Index Convention	6
7	PDF Operators ($b_T = 0$)	7
7.1	CG PDF (Coordinate Gauge)	7
7.2	GI PDF (Gauge Invariant)	7
8	Wilson Line Index Lists	7
8.1	TMD Wilson Line Indices	7
8.2	Reordering for Minimal Adjacent Differences	8
8.3	PDF Wilson Line Indices	8
9	Code Implementation Details	8
9.1	<code>__init__(parameters)</code>	8
9.2	<code>contract_2pt_pion</code>	9
9.3	<code>contract_qTMD_CG</code>	9
9.4	<code>contract_PDF</code>	9

9.5	<code>_contract_qTMD_one_shift</code>	9
9.6	<code>create_fw_prop_TMD_CG</code>	10
9.7	<code>create_fw_prop_PDF_GI</code>	10
9.8	<code>meson_backward_line</code>	10
10	Output Shape Convention and HDF5 Layout	11
10.1	Raw Contraction Shape	11
10.2	Explicit Transpose for HDF5 Write	11
10.3	Source Time Rolling	11
10.4	HDF5 Directory Layout	11
11	Boost Smearing Parameters	12
12	Comparison with Proton TMD	12
12.1	Flavor Dimension	12
12.2	Spin Polarization	12
12.3	Contraction Structure	12
12.4	Sequential Source	12
12.5	Summary Table	13
13	Summary of Key Formulae	13
A	Gamma Matrix Reference	14
B	Wilson Line Index Convention	14

1 Physics Overview

1.1 Transverse Momentum Dependent Distributions

Transverse momentum dependent (TMD) parton distribution functions and parton distribution functions (PDFs) encode the three-dimensional momentum structure of hadrons. The TMD correlator for a quark in a hadron is defined on the light cone,

$$\Phi_q(x, \mathbf{k}_\perp; P, S) = \int \frac{d\xi^- d^2\xi_\perp}{(2\pi)^3} e^{ik \cdot \xi} \langle P, S | \bar{\psi}(0) \mathcal{W}(0, \xi; \text{path}) \psi(\xi) | P, S \rangle \Big|_{\xi^+=0}, \quad (1)$$

where $\mathcal{W}(0, \xi; \text{path})$ is the staple-shaped Wilson line connecting the quark fields, P is the hadron momentum, and S its spin.

On the lattice, we work at equal time and replace the light-cone separation with a purely spatial displacement. The quasi-TMD correlator in the rest frame of the temporal direction is

$$\tilde{\Phi}_\Gamma(b_\perp, b_z, p^z; \tau) = \langle \pi(p^z) | \bar{\psi}(0) \Gamma \mathcal{W}_{\text{CG}}(0; b_\perp, b_z) \psi(\tau; b_\perp, b_z) | \pi(p^z) \rangle. \quad (2)$$

Here $\mathbf{b}_\perp = b_\perp \mathbf{e}_\perp$ is the transverse displacement (in the x or y direction), $b_z \mathbf{e}_z$ is the longitudinal displacement along the boost direction, τ is the operator insertion time, and $\mathcal{W}_{\text{CG}}$ denotes the coordinate-gauge Wilson line (or a true gauge-covariant staple for the operator with explicit gauge links). Γ is a Dirac matrix selecting the desired spin structure.

1.2 Equal-Time Correlators on the Lattice

On the Euclidean lattice the time coordinate is Wick-rotated: $t = i\tau$. The equal-time operator insertion means the quark and antiquark fields in the nonlocal bilinear are evaluated at the

same time slice τ (or equivalently the same Euclidean time t). The spatial displacement $\mathbf{b}$ is realized as a lattice shift operator acting on the quark propagator.

For the pion, which is a spin-zero meson, only specific Γ projections survive and the TMDs are spin-averaged. The pion qTMD distribution probes the transverse and longitudinal momentum correlations of the valence quark and antiquark within the pion.

2 Quasi-PDF and Quasi-TMD: LaMET Framework

Physical PDFs and TMDs are defined on the light cone, while lattice QCD computes correlation functions at equal time. Large momentum effective theory (LaMET) [1, 2] bridges the two: one computes a *quasi-distribution* on the lattice at finite hadron momentum p^z , then matches it to the physical distribution through a factorization formula

$$\tilde{q}(x, b_\perp, p^z; \mu) = \int \frac{dy}{|y|} C\left(\frac{x}{y}, \frac{\mu}{p^z}\right) q(y, \mu) + \mathcal{O}\left(\frac{\Lambda_{\text{QCD}}^2}{(p^z)^2}, \frac{\Lambda_{\text{QCD}}^2}{b_\perp^2}\right). \quad (3)$$

The pion boost momentum $\mathbf{p}_f = (0, 0, p^z)$ is a crucial parameter. Three regimes must be controlled:

- (i) The boost momentum must be large enough that power corrections $\Lambda_{\text{QCD}}^2/(p^z)^2$ are under control.
- (ii) The spatial displacement b_z must be small compared to the correlation length of the gauge links (to avoid large noise from Wilson line self-energy).
- (iii) The transverse displacement $b_\perp$ must be large enough to resolve non-perturbative transverse structure but small enough to avoid discretization effects.

In this workflow, the quasi-TMD is computed directly from the lattice three-point function with a nonlocal displacement operator inserted between the source and the fixed sink. The equal-time correlator is

$$\tilde{C}_3^\Gamma(q, b_\perp, b_z; \tau; p_f, t_{\text{sep}}) = \sum_{\mathbf{x}} \sum_{\mathbf{y}} e^{-i\mathbf{q} \cdot (\mathbf{x} - \mathbf{x}_0)} e^{-i\mathbf{p}_f \cdot (\mathbf{y} - \mathbf{x}_0)} \langle \pi(\mathbf{y}, t_{\text{sep}}) | \bar{\psi}(\mathbf{x}, \tau) \Gamma \mathcal{O}_{b_\perp, b_z} \psi(\mathbf{x}, \tau) | \pi(\mathbf{x}_0, 0) \rangle. \quad (4)$$

The momentum $\mathbf{p}_f$ is the fixed sink momentum that determines the pion boost; $\mathbf{q}$ is the Fourier momentum at the operator insertion; and the initial pion momentum $\mathbf{p}_i$ is related by momentum conservation at the insertion: $\mathbf{p}_f = \mathbf{p}_i + \mathbf{q}$ when the operator carries zero momentum (purely spatial displacement). In practice p_f is set to the desired boost and q scans all momentum modes of the lattice for the non-forward matrix element.

3 Two-Point Function

3.1 Physical Definition

The connected pion two-point function for source gamma Γ_{src} and sink gamma Γ_{sink} is

$$C_2^{\Gamma_{\text{sink}}}(p, t) = \sum_{\mathbf{x}} e^{-i\mathbf{p} \cdot (\mathbf{x} - \mathbf{x}_0)} \text{Tr}_{\text{spin, color}} \left[S_{\text{anti}}(\mathbf{x}, t; \mathbf{x}_0, 0) \Gamma_{\text{sink}} S_q(\mathbf{x}, t; \mathbf{x}_0, 0) \Gamma_{\text{src}} \right], \quad (5)$$

where $S_q(x, y)$ is the quark propagator from source y to sink x , and S_{anti} is the antiquark line constructed via γ_5 -hermiticity:

$$S_{\text{anti}}(x, y) = \gamma_5 S_q(x, y)^\dagger \gamma_5. \quad (6)$$

3.2 Source Gamma Convention

The default source gamma is `fixed_g5`, which sets $\Gamma_{\text{src}} = \gamma_5$ for all sink channels, corresponding to the standard pseudoscalar pion interpolator $\bar{\psi}\gamma_5\psi$. Three source modes are supported in the code (`_source_gamma_stack`):

- `fixed_g5`: All source gammas are γ_5 .
- `same_as_sink`: $\Gamma_{\text{src}} = \Gamma_{\text{sink}}$.
- `dagger_of_sink`: $\Gamma_{\text{src}} = \gamma_5 \Gamma_{\text{sink}}^\dagger \gamma_5$.

All 16 sink gamma matrices are scanned and stored, ordered as

$$\gamma_5, \gamma_T, \gamma_T\gamma_5, \gamma_X, \gamma_X\gamma_5, \gamma_Y, \gamma_Y\gamma_5, \gamma_Z, \gamma_Z\gamma_5, I, \Sigma_{XT}, \Sigma_{XY}, \Sigma_{XZ}, \Sigma_{YT}, \Sigma_{YZ}, \Sigma_{ZT}, \quad (7)$$

corresponding to PyQUDA gamma indices [15, 8, 7, 1, 14, 2, 13, 4, 11, 0, 9, 3, 5, 10, 6, 12].

3.3 Einsum Contraction Pattern

The array layout of a PyQUDA propagator is

$$\text{prop.data}[w, t, z, y, x_{\text{cb}}, \text{spin}_{\text{sink}}, \text{spin}_{\text{src}}, \text{color}_{\text{sink}}, \text{color}_{\text{src}}], \quad (8)$$

where w is the even-odd parity index (checkerboard), t, z, y, x_{cb} are space-time coordinates, and the remaining four indices carry spin and color.

The antiquark line is constructed as (`_meson_backward_line`):

$$\text{bw_prop}[w, t, z, y, x_{\text{cb}}, j, i, b, a] = (\gamma_5)_{ki} \text{prop.data}[w, t, z, y, x_{\text{cb}}, l, a, b]^* (\gamma_5)_{lj}. \quad (9)$$

The contraction proceeds in two einsum steps:

1. Attach sink gamma to backward line:

$$\text{bw_prop}_\Gamma[g, w, t, z, y, x_{\text{cb}}, j, m, c, f] = \text{bw_prop}[w, t, z, y, x_{\text{cb}}, j, i, c, f] (\Gamma_g)_{im}. \quad (10)$$

2. Full contraction with source gamma and color/spin trace:

$$\text{corr_local}[g, w, t, z, y, x_{\text{cb}}] = \text{bw_prop}_\Gamma[g, w, t, z, y, x_{\text{cb}}, j, i, a, b] \text{prop.f.data}[w, t, z, y, x_{\text{cb}}, l, b, a] (\Gamma_{\text{src}})_{lj}. \quad (11)$$

The momentum phase is applied via a Fourier-phase tensor `phases`[$q, w, t, z, y, x_{\text{cb}}$] generated by `MomentumPhase.getPhases` with momentum arguments p_{xyz} and source position x_0 . The phase convention in the code uses $e^{-i\mathbf{p}\cdot(\mathbf{x}-\mathbf{x}_0)}$: for the two-point function, negated three-momenta $[-p_x, -p_y, -p_z]$ are passed to the phase generator, so that

$$\text{phases}[q, w, t, z, y, x_{\text{cb}}] \hat{=} e^{-i\mathbf{p}_q\cdot(\mathbf{x}-\mathbf{x}_0)}. \quad (12)$$

The final momentum-projected correlator is

$$\text{corr}[g, q, t] = \sum_{w, z, y, x_{\text{cb}}} \text{phases}[q, w, t, z, y, x_{\text{cb}}] \text{corr_local}[g, w, t, z, y, x_{\text{cb}}]. \quad (13)$$

3.4 Boosted Smearing

Before contraction, both the forward propagator `prop_f` and the backward propagator `prop_b` are boosted-smearred at the sink using separate boost parameters `pos_boost` and `neg_boost`:

$$\text{prop_f} \leftarrow \text{boosted_smearing}(\text{prop_f}, w, \text{pos_boost}), \quad (14)$$

$$\text{prop_b} \leftarrow \text{boosted_smearing}(\text{prop_b}, w, \text{neg_boost}). \quad (15)$$

Boosted smearing is a gauge-covariant Gaussian convolution in momentum space with an injected momentum shift: the fermion field is Fourier-transformed (assuming identity gauge, i.e., already fixed to a smooth gauge), multiplied by a momentum-shifted Gaussian kernel $\exp(-\frac{|\mathbf{r}|^2}{2w^2} + i\mathbf{k}_{\text{boost}} \cdot \mathbf{r})$, and inverse-transformed back to position space.

4 Three-Point qTMD Function (Pre-Sequential-Trick)

4.1 Physical Correlator

The connected pion qTMD three-point function, before applying the sequential source trick, reads

$$C_3^\Gamma(q, b_\perp, b_z; \tau; p_f, t_{\text{sep}}) = \sum_{\mathbf{x}} \sum_{\mathbf{y}} e^{-i\mathbf{q} \cdot (\mathbf{x} - \mathbf{x}_0)} e^{-i\mathbf{p}_f \cdot (\mathbf{y} - \mathbf{x}_0)} \times \text{Tr}_{\text{spin, color}} \left[S_{\text{anti}}(\mathbf{y}, t_{\text{sep}}; \mathbf{x}_0, 0) \Gamma_{\text{sink}} S_q(\mathbf{y}, t_{\text{sep}}; \mathbf{x}, \tau) \Gamma_{\text{ins}} \mathcal{O}_{b_\perp, b_z} S_q(\mathbf{x}, \tau; \mathbf{x}_0, 0) \Gamma_{\text{src}} \right]. \quad (16)$$

Here $\mathcal{O}_{b_\perp, b_z}$ is the nonlocal displacement operator acting on the forward quark line. Unlike the pion electromagnetic form factor (EMFF), where the insertion is a local current at a single space-time point, the qTMD insertion is nonlocal in space: the quark field at $(\tau, \mathbf{x})$ is shifted by $(b_\perp \mathbf{e}_\perp + b_z \mathbf{e}_z)$. This is the defining difference from the EMFF three-point function.

The momenta satisfy $\mathbf{p}_i$ (initial), $\mathbf{p}_f$ (final sink), and $\mathbf{q}$ (insertion) with $\mathbf{q} = \mathbf{p}_f - \mathbf{p}_i$ in the forward matrix element.

4.2 Why the Sequential Trick is Needed

Computing Eq. (16) directly would require a separate propagator inversion for every $(\mathbf{q}, b_\perp, b_z, \tau)$ combination, which is prohibitively expensive. The fixed-sink sequential source method absorbs the entire sink sum (over $\mathbf{y}$) into a single propagator inversion, after which the insertion-side contraction becomes a simple two-point-like correlation.

5 Sequential Source

5.1 Meson Sequential Source Construction

The meson fixed-sink sequential source is built in `create_meson_bw_seq_pyquda`. Given the sink-smearred propagator `prop` (already boosted-smearred with `neg_boost`), the source position x_0 , the sink momentum p_f , the sink-source separation t_{sep} , and the sink Dirac matrix Γ_{sink} , the right-hand side is placed on the sink time slice $t_{\text{sink}} = t_0 + t_{\text{sep}}$:

$$\eta_{\text{seq}}(\mathbf{y}; p_f, t_{\text{sep}}) = \delta_{t_y, t_{\text{sink}}} \Phi_{p_f}(\mathbf{y} - \mathbf{x}_0) \Gamma_{\text{seq}} S_q(\mathbf{y}, t_{\text{sink}}; \mathbf{x}_0, 0), \quad (17)$$

$$\Gamma_{\text{seq}} = \gamma_5 \Gamma_{\text{sink}}^\dagger \gamma_5. \quad (18)$$

The phase function $\Phi_{p_f}(\mathbf{r}) = e^{-i\mathbf{p}_f \cdot \mathbf{r}}$ is generated by `MomentumPhase.getPhases` with the same sign convention as the two-point function. The time slice restriction is enforced by `pyquda_utils.source.sequential12`, which masks all time slices except t_{sink} .

The sequential propagator solves

$$D S_{\text{seq}}(x; x_0) = \eta_{\text{seq}}(x), \quad (19)$$

where D is the Wilson-clover Dirac operator. After inversion, γ_5 -hermiticity yields the backward sequential line used in contraction:

$$S_{\text{seq}}^{\text{anti}}(x, x_0; p_f, t_{\text{sep}}) = \gamma_5 S_{\text{seq}}(x, x_0)^\dagger \gamma_5. \quad (20)$$

5.2 Sink-Summed Three-Point Contraction

With the sequential propagator, the sink sum is absorbed and the three-point function reduces to a two-point-like form:

$$C_3^\Gamma(q, b_\perp, b_z; \tau; p_f, t_{\text{sep}}) = \sum_{\mathbf{x}} e^{-i\mathbf{q} \cdot (\mathbf{x} - \mathbf{x}_0)} \text{Tr}_{\text{spin,color}} \left[S_{\text{seq}}^{\text{anti}}(\mathbf{x}, \tau; \mathbf{x}_0, 0) \Gamma_{\text{ins}} \mathcal{O}_{b_\perp, b_z} S_q(\mathbf{x}, \tau; \mathbf{x}_0, 0) \Gamma_{\text{src}} \right]. \quad (21)$$

The insertion gamma Γ_{ins} scans all 16 Dirac structures (the same set as in Sec. 3), while Γ_{src} is typically `fixed_g5` and Γ_{sink} is set by the application (e.g., γ_5 for the pseudoscalar pion).

The einsum contraction pattern in `_contract_qTMD_one_shift` mirrors the two-point function:

$$\text{sink_inserted}[g, w, t, z, y, x_{\text{cb}}, j, m, c, f] = S_{\text{seq}}^{\text{anti}}[w, t, z, y, x_{\text{cb}}, j, i, c, f] (\Gamma_g)_{im}, \quad (22)$$

$$\text{corr_local}[g, w, t, z, y, x_{\text{cb}}] = \text{sink_inserted}[g, w, t, z, y, x_{\text{cb}}, j, i, a, b] (\text{shifted_prop})[w, t, z, y, x_{\text{cb}}] \quad (23)$$

Here `shifted_prop` is the forward propagator with the nonlocal displacement operator already applied (see Sec. 6).

6 CG qTMD Operator

6.1 Coordinate-Gauge Displacement

For the CG (coordinate-gauge) qTMD path, the nonlocal operator is approximated by a simple lattice shift, with *no explicit gauge link* inserted:

$$\mathcal{O}_{b_\perp, b_z} S_q(x, x_0) = S_q(x + b_\perp \mathbf{e}_\perp + b_z \mathbf{e}_z, x_0). \quad (24)$$

This corresponds to working in a gauge where the spatial links are (at least approximately) set to unity, so the Wilson line is trivial. The physical interpretation is that the gauge has been fixed to a smooth gauge (e.g., Coulomb or Landau gauge) before the correlation function is computed.

6.2 Transverse Direction Scan

The transverse displacement direction $\mathbf{e}_\perp$ is scanned over the two transverse lattice axes:

- Direction 0: $\mathbf{e}_\perp = \mathbf{e}_x$ (stored as `b_X` in HDF5).
- Direction 1: $\mathbf{e}_\perp = \mathbf{e}_y$ (stored as `b_Y` in HDF5).

The code contracts both directions separately with independent propagator copies so that successive shifts remain small and error accumulation is minimized.

6.3 Wilson Line Index Convention

Each displacement is labeled by a four-component Wilson line index:

$$\text{W_index} = [b_T, b_z, \eta, \text{direction}], \quad (25)$$

where:

- b_T : transverse displacement in lattice units (0 to `b_T`).
- b_z : longitudinal displacement in lattice units (positive and negative, up to $\pm \text{b_z}$).
- η : staple length parameter (set to 0 for straight-line paths in current implementation).
- direction: 0 for $\mathbf{e}_x$, 1 for $\mathbf{e}_y$.

The incremental shift operation in `create_fw_prop_TMD_CG` computes

$$\Delta b_T = \text{round}(b_T^{\text{current}} - b_T^{\text{previous}}), \quad \Delta b_z = \text{round}(b_z^{\text{current}} - b_z^{\text{previous}}), \quad (26)$$

and applies `prop.f.shift( $\Delta b_T$ , direction).shift( $\Delta b_z$ , 2)`, where direction 2 is the z -axis. The round function handles cases where b_T or b_z could be non-integer (though in current use they are integers).

7 PDF Operators ($\mathbf{b}_T = 0$)

The PDF path is the straight- z special case with $b_\perp = 0$:

$$\mathbf{b} = b_z \mathbf{e}_z. \quad (27)$$

Two variants are supported, selected by the boolean parameter `gauge_invariant`:

7.1 CG PDF (Coordinate Gauge)

The CG PDF operator is identical in spirit to the CG qTMD operator (Sec. 6) but restricted to the z direction:

$$\mathcal{O}_{b_z}^{\text{CG}} S_q(x, x_0) = S_q(x + b_z \mathbf{e}_z, x_0). \quad (28)$$

No gauge links are included. This is implemented by the same `create_fw_prop_TMD_CG` method, which applies a lattice shift in the z direction (direction index 2).

7.2 GI PDF (Gauge Invariant)

The GI PDF operator inserts explicit gauge-covariant shifts that build up the Wilson line incrementally. For a unit step in the $+z$ direction:

$$\psi'(x) = U_z(x) \psi(x + \hat{z}), \quad (29)$$

and for a unit step in the $-z$ direction:

$$\psi'(x) = U_{-z}(x) \psi(x - \hat{z}) = U_z^\dagger(x - \hat{z}) \psi(x - \hat{z}). \quad (30)$$

The implementation in `create_fw_prop_PDF_GI` applies the gauge-covariant derivative `gauge.pure_gauge.dir` spin-color by spin-color:

- `covDev(fermion, 2)`: covariant shift in $+z$ ($\Delta b_z = +1$).
- `covDev(fermion, 6)`: covariant shift in $-z$ ($\Delta b_z = -1$).

The PyQUDA direction convention is: 0, 1, 2, 3 for $+x, +y, +z, +t$ and 4, 5, 6, 7 for $-x, -y, -z, -t$.

An important implementation detail is that only nearest-neighbor ($\Delta b_z = \pm 1$) incremental changes are supported. If the ordered Wilson line list contains a jump larger than one lattice unit, the code raises a `ValueError` rather than silently producing an incorrect path. This constraint is satisfied by construction of the Wilson line index list (see Sec. 8).

8 Wilson Line Index Lists

8.1 TMD Wilson Line Indices

The method `create_TMD_Wilsonline_index_list_CG` returns two lists: `index_list_trans0` (transverse direction 0, i.e., x) and `index_list_trans1` (transverse direction 1, i.e., y).

For a given maximum displacement ($b_T^{\text{max}}, b_z^{\text{max}}$), the index pairs are enumerated as:

```

for bz in 0 .. b_z:
    for bT in 0 .. b_T:
        add [bT, bz, 0, 0]    and    [bT, bz, 0, 1]
        if bz != 0:
            add [bT, -bz, 0, 0]    and    [bT, -bz, 0, 1]

```

This generates all (b_T, b_z) pairs in the first quadrant and mirror- z half of the (b_T, b_z) plane. Both positive and negative b_z are included to capture the full Fourier range for the z -direction momentum projection.

8.2 Reordering for Minimal Adjacent Differences

After enumeration, `_reorder_wilson_indices` reorders each list to minimize the maximum absolute difference in (b_T, b_z) between adjacent indices. The algorithm:

1. Sort by (b_T, b_z) lexicographically.
2. Walk through the sorted list; if the jump from index i to $i + 1$ exceeds one lattice unit in either coordinate, search ahead for a closer candidate and swap it into position $i + 1$.

This greedy reordering reduces the cumulative lattice displacement traversed during incremental propagator shifts, which minimizes numerical noise accumulation and improves code efficiency since smaller shifts are cheaper in the PyQUDA propagator implementation.

8.3 PDF Wilson Line Indices

The method `create_PDF_Wilsonline_index_list` returns a single list (because $b_T = 0$ always):

```

for bz in 0 .. b_z:
    add [0, bz, 0, 0]
for bz in 0 .. b_z:    # skip bz=0 (already added above)
    if bz != 0:
        add [0, -bz, 0, 0]

```

The list starts at $b_z = 0$ (the unshifted propagator) and incrementally walks out to $\pm b_z^{\max}$ in unit steps. Since b_T is identically zero and the b_z changes are by construction ± 1 or 0 (resetting to $b_z = 0$ between the positive and negative branches), no reordering is needed.

For the GI PDF path, the reset to $b_z = 0$ between branches is essential: it restarts the propagator from the original (unshifted) copy and builds the negative- b_z branch from scratch, avoiding cumulative gauge-link noise from traversing the entire length $+b_z^{\max}$ to $-b_z^{\max}$.

9 Code Implementation Details

The core implementation resides in `pyquda_measurement_utils/pion_qTMD_vibe_develop.py`, in the class `pion_TMD`. Below we document each major method.

9.1 `__init__(parameters)`

Constructs the measurement object from a parameter dictionary with keys:

- `eta`: staple length parameter (list, typically `[0]`).
- `b_z`, `b_T`: maximum longitudinal and transverse displacements.
- `pf`: sink boost momentum $[p_x, p_y, p_z, p_t]$.
- `qext`, `qext_PDF`: insertion momentum lists for qTMD and PDF respectively.
- `p_2pt`: momentum list for the two-point function.
- `width`: Gaussian smearing width.

- `pos_boost`, `neg_boost`: boosted smearing momentum boosts for quark and antiquark lines.
- `t_insert`: sink-source separation t_{sep} .
- `save_propagators`: whether to save propagators to disk.

9.2 `contract_2pt_pion`

```
def contract_2pt_pion(self, latt_info, prop_f, prop_b, phases, tag,
                     src_gamma="fixed_g5"):
```

Workflow:

1. Boosted-smear `prop_f` (with `pos_boost`) and `prop_b` (with `neg_boost`).
 2. Build sink gamma stack (`_gamma_stack`) and source gamma stack (`_source_gamma_stack`).
 3. Construct antiquark line via `_meson_backward_line` (einsum with γ_5 front and back).
 4. Attach sink gammas to backward line: einsum "wtzyxjicf,gim->gwtzyxjmcf".
 5. Contract with forward propagator and source gammas: einsum "gwtzyxjiab,wtzyxilba,glj->gwtzyx".
 6. Fourier-project with `phases`: einsum "qwtzyx,gwtzyx->gqt".
 7. Gather across MPI ranks (`core.gatherLattice`).
 8. Save to HDF5 via `save_proton_c2pt_hdf5`.
- The output shape is `[gamma, momentum, time]`.

9.3 `contract_qTMD_CG`

```
def contract_qTMD_CG(self, latt_info, prop_f, seq_bw_prop, phases,
                     W_index_list_dir0, W_index_list_dir1,
                     src_gamma="fixed_g5"):
```

Workflow:

1. Prepare gamma stacks (Γ_g and Γ_{src}) and phases.
2. Construct sequential backward line `seq_bw_line =  $\gamma_5$  seq_bw_prop $^\dagger \gamma_5$` .
3. For direction 0 (x): iterate over `W_index_list_dir0`, incrementally shifting `tmd_forward_prop_dir0` using `create_fw_prop_TMD_CG` and contracting at each step via `_contract_qTMD_one_shift`.
4. Reset and repeat for direction 1 (y) with `W_index_list_dir1`.
5. Stack all results into a numpy array.

Each Wilson index produces an output of shape `[gamma, momentum, time]`; stacking across all N_W Wilson indices yields shape `[N_W, gamma, momentum, time]`.

9.4 `contract_PDF`

```
def contract_PDF(self, latt_info, gauge, prop_f, seq_bw_prop, phases,
                 W_index_list, src_gamma="fixed_g5",
                 gauge_invariant=True):
```

Same structure as `contract_qTMD_CG` but:

- Uses a single `W_index_list` (no transverse direction separation).
- Resets the forward propagator to the original copy whenever b_z returns to 0 (handling the split between positive and negative branches).
- Calls `create_fw_prop_PDF_GI` when `gauge_invariant=True` or `create_fw_prop_TMD_CG` when `gauge_invariant=False`.

9.5 `_contract_qTMD_one_shift`

```
def _contract_qTMD_one_shift(self, seq_bw_line, shifted_prop,
                             sink_gamma_ls, source_gamma_ls, phases):
```

The core single-Wilson-index contraction:

1. `sink_inserted`: `einsum "wtzyxjicf,gim->gwtzyxjmcf"` (attach Γ_g to sequential backward line).
2. `corr_local`: `einsum "gwtzyxjiab,wtzyxilba,glj->gwtzyx"` (contract Γ_g line, shifted forward propagator, Γ_{src}).
3. Momentum projection: `einsum "qwtzyx,gwtzyx->gqt"`.
4. MPI gather: `core.gatherLattice`.

The index conventions are:

- g : gamma index (0–15).
- q : momentum index.
- w : checkerboard parity.
- t, z, y, x_{cb} : space-time coordinates.
- i, j, k, l, m : spin indices.
- a, b, c, f : color indices.

9.6 create_fw_prop_TMD_CG

```
def create_fw_prop_TMD_CG(self, prop_f, W_index, W_index_previous):
    dbT = current_b_T - previous_b_T
    dbz = current_bz - previous_bz
    return prop_f.shift(round(dbT), trans_dir).shift(round(dbz), 2)
```

Applies incremental lattice shifts to propagate the forward propagator from one Wilson index to the next. The z -direction is always index 2 (the third spatial axis in PyQUDA's convention).

9.7 create_fw_prop_PDF_GI

```
def create_fw_prop_PDF_GI(self, gauge, prop_f, W_index, W_index_previous):
    for spin in range(4):
        for color in range(3):
            fermion = prop_f.getFermion(spin, color)
            if current_bz - previous_bz == 1:
                fermion = gauge.pure_gauge.covDev(fermion, 2) # +z
            elif current_bz - previous_bz == -1:
                fermion = gauge.pure_gauge.covDev(fermion, 6) # -z
            prop_f.setFermion(fermion, spin, color)
```

Applies a gauge-covariant unit shift spin-color by spin-color. The covariant derivative `covDev` parallel-transportes the fermion from the neighboring lattice site back to the current site using the gauge link, implementing Eq. (7) of Sec. 7.

9.8 _meson_backward_line

```
def _meson_backward_line(prop):
    return xp.einsum("ij,wtzyxilab,kl->wtzyxkjba",
                    gamma5, prop.data.conj(), gamma5)
```

Implements the γ_5 -hermiticity transformation:

$$(S^{\text{anti}})^{kj}_{ba}(\mathbf{x}) = (\gamma_5)_{ik} (S_{qab}^{li}(\mathbf{x}))^* (\gamma_5)_{lj}. \quad (31)$$

Note that both color and spin indices are transposed by the conjugation, and γ_5 acts on both the source and sink spin spaces.

10 Output Shape Convention and HDF5 Layout

10.1 Raw Contraction Shape

The contraction routines naturally produce arrays of shape

$$[\text{Wilson_index}, \text{gamma}, \text{momentum}, \text{time}]. \quad (32)$$

10.2 Explicit Transpose for HDF5 Write

The Perlmutter application explicitly transposes the contraction array before calling the HDF5 writer:

$$\text{pion_TMDs} \leftarrow \text{np.transpose}(\text{pion_TMDs}, (0, 2, 1, 3)), \quad (33)$$

yielding shape `[Wilson_index, momentum, gamma, time]`. This transpose is critical: `save_qTMD_pion_hdf5` (which delegates to `save_qTMD_proton_hdf5_noRoll`) expects the momentum dimension to precede the gamma dimension. Without this transpose, a single-gamma HDF5 file could be written with a momentum list interpreted as gamma labels, producing silently corrupted output.

10.3 Source Time Rolling

The two-point function data is time-rolled by the source time t_0 in `save_proton_c2pt_hdf5`:

$$C_2(t) \leftarrow C_2(t + t_0 \bmod L_t), \quad (34)$$

so that $t = 0$ in the saved data corresponds to the source time slice.

For the three-point function, the application rolls before calling the writer:

$$\text{pion_TMDs} \leftarrow \text{np.roll}(\text{pion_TMDs}, -\text{pos}[3], \text{axis}=-1), \quad (35)$$

and additionally truncates to $\tau \in [0, t_{\text{sep}}]$:

$$\text{pion_TMDs} \leftarrow \text{pion_TMDs}[:, :, :, :t_{\text{insert}}+2]. \quad (36)$$

10.4 HDF5 Directory Layout

The HDF5 file hierarchy for the three-point function is:

`SS/<gamma>/PX..PY..PZ../<b_X or b_Y>/eta0/bT<bT>/bz<bz>`

where:

- **SS**: sequential-source group.
- **<gamma>**: one of the 16 gamma labels (5, T, T5, ..., SZT).
- **PX..PY..PZ..**: insertion momentum components.
- **b_X or b_Y**: transverse displacement direction.
- **eta0**: staple length parameter (currently 0).
- **bT<bT>**: transverse displacement magnitude.
- **bz<bz>**: longitudinal displacement (dataset with time-series data).

For the PDF path ($b_T = 0$), the datasets appear under `b_X/eta0/bT0/` regardless of the lack of transverse displacement, maintaining structural compatibility with the qTMD layout.

11 Boost Smearing Parameters

The boosted smearing parameters `pos_boost` and `neg_boost` are three-component vectors $\mathbf{k}_{\text{boost}} = (k_x, k_y, k_z)$ in units of $2\pi/L$. They control the momentum shift injected into the smearing kernel for the quark (forward) and antiquark (backward) lines respectively.

In the Gaussian smearing kernel

$$K(\mathbf{r}) = \exp\left(-\frac{|\mathbf{r}|^2}{2w^2} + i \mathbf{k}_{\text{boost}} \cdot \mathbf{r}\right), \quad (37)$$

the boost momentum shifts the peak of the smearing kernel in momentum space, effectively selecting quark fields that carry momentum $\mathbf{k}_{\text{boost}}$. This improves the overlap of the interpolating operator with a moving pion state and is essential for achieving good signal at finite boost p_f .

In the current pion qTMD workflow, both `pos_boost` and `neg_boost` are set to `0`, and the pion boost is provided solely by the sink momentum p_f encoded in the sequential source. The parameters are retained for future workflows that may use non-zero boost smearing to improve overlap with boosted pion states.

12 Comparison with Proton TMD

The pion qTMD code (`pion_qTMD_vibe_develop.py`) has the same structural organization as the proton qTMD code (`proton_qTMD_pyquda.py`). The key differences arise from the meson- vs.- baryon nature of the hadron:

12.1 Flavor Dimension

- **Proton:** Three quark lines (u, u, d), requiring separate sequential sources for up-quark and down-quark insertions. The flavor index is an explicit dimension of the output data.
- **Pion:** Two quark lines (quark and antiquark) treated symmetrically. There is no flavor dimension; a single sequential source suffices.

12.2 Spin Polarization

- **Proton:** Spin-1/2 particle with polarization dependence. The sequential source construction includes spin projection matrices $P_p S_{zp}, P_p S_{zm}, P_p S_{xp}, P_p S_{xm}, P_p^{\text{unpol}}$ via `PolProjections`.
- **Pion:** Spin-0 particle. No spin polarization is needed. The sequential source is a simple meson backward line (Eq. 17) without spin projection.

12.3 Contraction Structure

- **Proton:** Uses baryon epsilon-tensor contractions ($\epsilon_{abc}\epsilon_{def}$) to form the color-singlet proton state. The two-point function involves two contraction terms (direct and exchange), and the three-point function involves separate diquark contractions (`down_quark_insertion_pyquda`, `up_quark_insertion_pyquda`).
- **Pion:** Uses meson contraction (simple color trace) via `_meson_backward_line` for the antiquark line. The pion is a color singlet $\delta_{ab}\bar{\psi}_a\psi_b$, so the contraction is a single term with no exchange contributions.

12.4 Sequential Source

- **Proton:** `create_bw_seq_pyquda` builds the baryon sequential source, which involves diquark contractions, time slicing, momentum phase insertion, γ_5 -conjugation, boosted smearing, inversion, and final spin projection.

- **Pion:** `create_meson_bw_seq_pyquda` builds the meson sequential source, which is considerably simpler: time-slice the propagator, apply $\Gamma_{\text{seq}} = \gamma_5 \Gamma_{\text{sink}}^\dagger \gamma_5$ to the source spin index, multiply by the momentum phase, invert, and return the propagator.

12.5 Summary Table

Feature	Pion qTMD	Proton qTMD
Hadron type	Meson (spin-0)	Baryon (spin-1/2)
Quark lines	2 ($q + \bar{q}$)	3 (u, u, d)
Flavor dimension	None	2 (up, down)
Spin polarization	None	5 projections
Contraction	<code>_meson_backward_line</code>	ϵ baryon contr.
Sequential source	<code>create_meson_bw_seq</code>	<code>create_bw_seq</code>
Wilson line ops	CG qTMD, CG/PDF, GI/PDF	CG qTMD, CG/PDF, GI/PDF
HDF5 layout	Same as proton	Standard

13 Summary of Key Formulae

For quick reference, the central contracted expressions are:

Two-point (Eq. 5):

$$C_2^g(p, t) = \sum_x e^{-ip \cdot (x - x_0)} \text{Tr}_{\text{sp, col}} [(\gamma_5 S_q^\dagger \gamma_5)(g) \Gamma_g S_q(x, x_0) \Gamma_{\text{src}}].$$

Sink-summed three-point (Eq. 21):

$$C_3^g(q, b, \tau; p_f, t_{\text{sep}}) = \sum_x e^{-iq \cdot (x - x_0)} \text{Tr}_{\text{sp, col}} [S_{\text{seq}}^{\text{anti}}(x, \tau) \Gamma_g \mathcal{O}_b S_q(x, \tau) \Gamma_{\text{src}}].$$

Meson sequential source (Eqs. 17, 18):

$$\eta_{\text{seq}}(y) = \delta_{t_y, t_{\text{sink}}} \Phi_{p_f}(y - x_0) (\gamma_5 \Gamma_{\text{sink}}^\dagger \gamma_5) S_q(y, x_0).$$

CG displacement (Eq. 24):

$$\mathcal{O}_{b_\perp, b_z} S_q(x, x_0) = S_q(x + b_\perp e_\perp + b_z e_z, x_0).$$

GI covariant shift (Sec. 7, unit $+z$):

$$\psi'(x) = U_z(x) \psi(x + \hat{z}), \quad (\text{covDev}(\text{fermion}, 2)).$$

A Gamma Matrix Reference

Index	Label	Description
0	5	γ_5
1	T	γ_T (temporal)
2	T5	$\gamma_T \gamma_5$
3	X	γ_X
4	X5	$\gamma_X \gamma_5$
5	Y	γ_Y
6	Y5	$\gamma_Y \gamma_5$
7	Z	γ_Z
8	Z5	$\gamma_Z \gamma_5$
9	I	Identity
10	SXT	$\Sigma_{XT} = \frac{i}{2} [\gamma_X, \gamma_T]$
11	SXY	Σ_{XY}
12	SXZ	Σ_{XZ}
13	SYT	Σ_{YT}
14	SYZ	Σ_{YZ}
15	SZT	Σ_{ZT}

B Wilson Line Index Convention

Field	Values	Meaning
b_T	$0, 1, \dots, b_T^{\max}$	Transverse displacement
b_z	$-b_z^{\max}, \dots, +b_z^{\max}$	Longitudinal displacement
η	0 (current)	Staple length parameter
direction	0 or 1	0 for x , 1 for y

References

- [1] X. Ji, *Parton Physics on a Euclidean Lattice*, Phys. Rev. Lett. **110**, 262002 (2013), arXiv:1305.1539 [hep-ph].
- [2] X. Ji, *Parton Physics from Large-Momentum Effective Field Theory*, Sci. China Phys. Mech. Astron. **57**, 1407–1412 (2014), arXiv:1404.6680 [hep-ph].
- [3] X. Ji, Y.-S. Liu, Y. Liu, J.-H. Zhang, and Y. Zhao, *Large-Momentum Effective Theory*, Rev. Mod. Phys. **93**, no. 3, 035005 (2021), arXiv:2004.03543 [hep-ph].
- [4] A. V. Radyushkin, *Quasi-parton distribution functions, momentum distributions, and pseudo-parton distribution functions*, Phys. Rev. D **96**, 034025 (2017), arXiv:1705.01488 [hep-ph].
- [5] K. Orginos, A. Radyushkin, J. Karpie, and S. Zafeiropoulos, *Lattice QCD exploration of parton pseudo-distribution functions*, Phys. Rev. D **96**, 094503 (2017), arXiv:1706.05373 [hep-ph].